

Lecture 4: Time Series & Recommender Systems

41-501
Applied AI

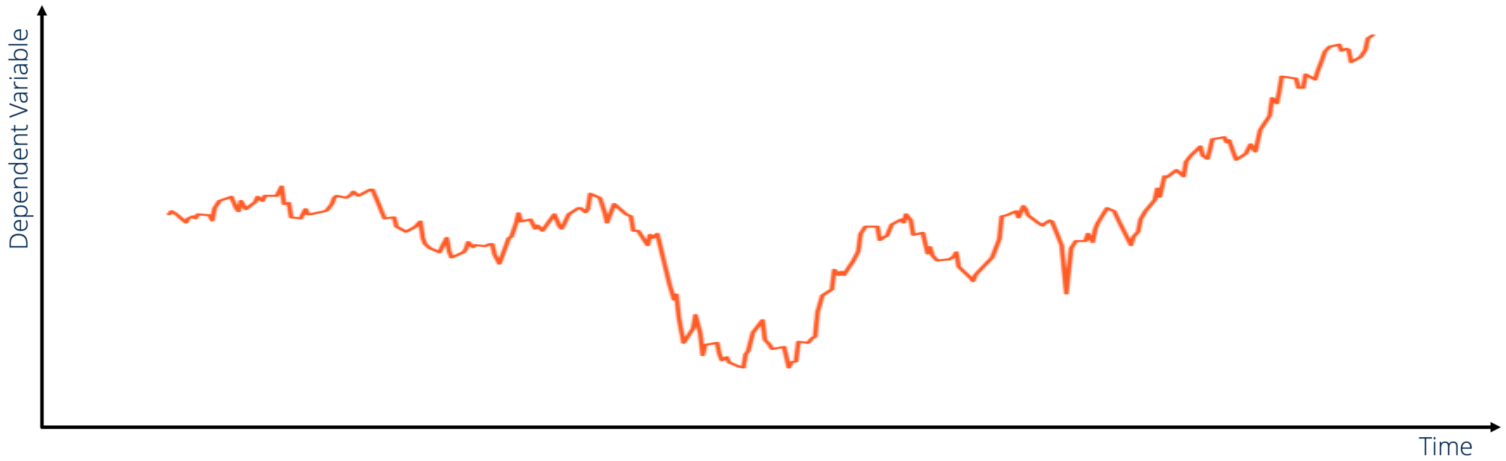
etienne@cmkl.ac.th

Content

- Time Series
- Recommender Systems
- Recap & Lab



Time Series



- Data points are ordered in time -> order matters
- Observations are not independent (unlike tabular ML from Week 2)
- Nearby points tend to be similar (autocorrelation)
- You can't randomly shuffle your training set



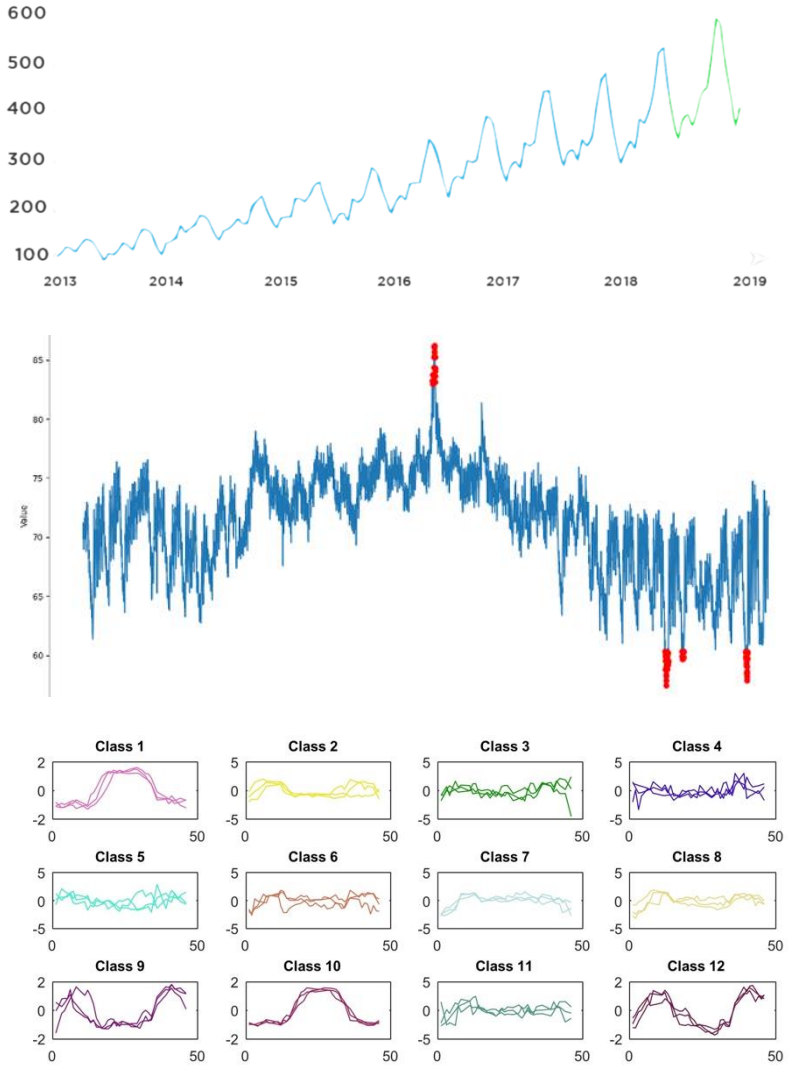
Time Series Everywhere

- **Finance**: stock prices, revenue forecasting
- **Energy**: grid load, solar output prediction
- **Retail**: demand forecasting, inventory planning
- **Manufacturing**: predictive maintenance, sensor data
- **Health**: heart rate, patient vitals monitoring



Three Types

1. **Forecasting**: predict future values (e.g., sales next month)
2. **Anomaly Detection**: spot unusual patterns (e.g., server outage)
3. **Classification**: label entire sequences (e.g., activity recognition from accelerometer)

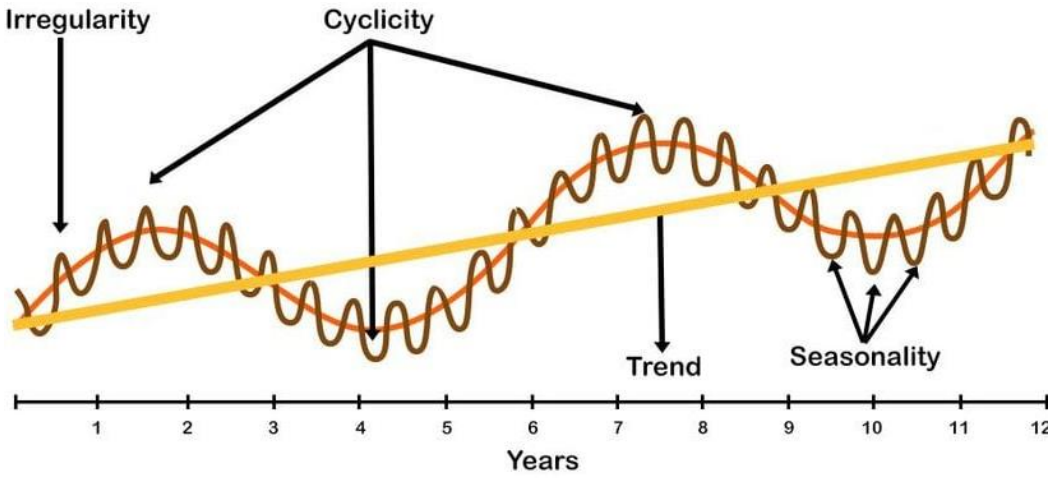


Decomposition

Any time series can be thought of as:

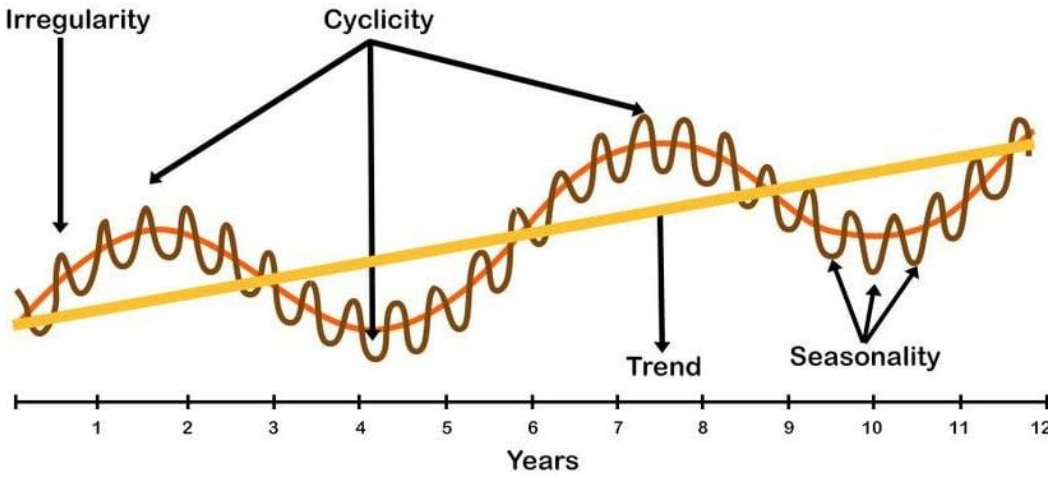
Observed = Trend + Seasonality + Residual

- Trend: long-term direction (up, down, flat)
- Seasonality: repeating patterns (daily, weekly, yearly)
- Residual: what's left (noise and irregular events)



Trend

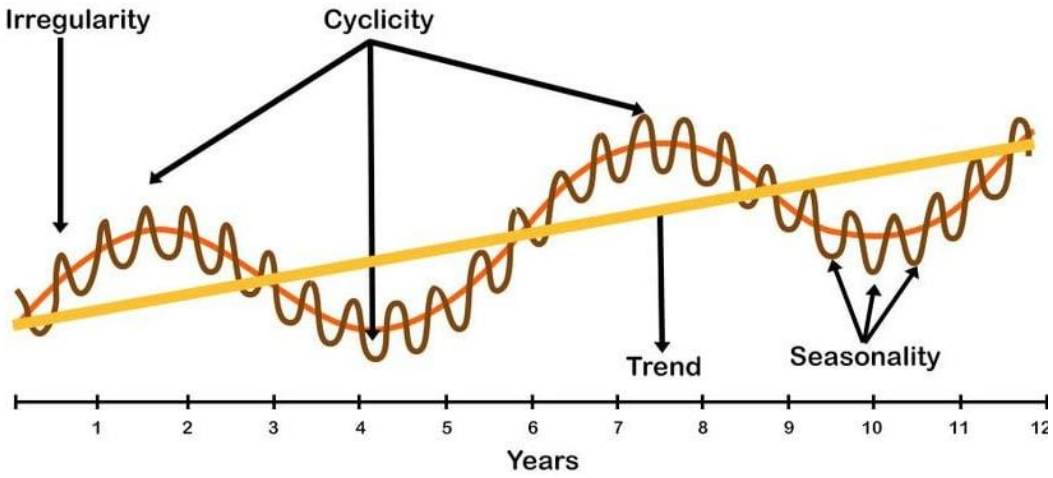
- Long-term increase or decrease
- Can be linear, polynomial, or step-wise
- **Examples**: growing user base, gradual equipment degradation



Seasonality

Repeating cycles at known fixed period:

- Daily: restaurant traffic peaks at lunch/dinner
- Weekly: e-commerce peaks on weekends
- Yearly: heating demand in winter



→ Multiple seasonalities can coexist (daily + weekly + yearly)

Stationarity

- A series is stationary if its statistical properties (mean, variance) don't change over time
- Most real-world data is not stationary
- Many methods assume or require stationarity

→ Common fix: differencing (subtract previous value)



The Landscape of Methods

Classical	ML-Based	Deep Learning
ARIMA, Exponential Smoothing	Gradient Boosting on engineered features	LSTMs, Temporal CNNs, Transformers
Strong baselines, interpretable	Flexible, handles many features	Learns patterns automatically, needs lots of data



Features: Lag

- Use previous values as input features
- lag_1 = value at t−1, lag_7 = value at t−7, etc.
- Captures autocorrelation directly
- Example (daily retail sales):

date	sales	lag_1	lag_7
Sep 18	120	115	105
Sep 19	130	120	110



Features: Rolling / Window

- Summaries over a sliding window
- rolling_mean_7: average of last 7 days
- rolling_std_7: volatility of last 7 days
- rolling_min, rolling_max

Smooths out noise, captures local trends



Features: Calendar

- day_of_week (0-6), month (1-12), is_weekend, is_holiday
- hour_of_day for intraday data
- quarter, week_of_year
- Encodes seasonality explicitly

These are often the most powerful features in business forecasting



Putting It Together

- Original data: just a date column and a target column
- After feature engineering:

date	lag_1	lag_7	roll_mean_7	roll_std_7	dow	month	is_weekend	target

- → Feed this into XGBoost / LightGBM → get a forecast



Train/Test Split: Don't Shuffle!

- In tabular ML: random split is fine
- In time series: you must split by time
- Train on past, test on future (always!)
- Otherwise, you're leaking future information into training



Outlook: DL Approaches

- **LSTMs / GRUs:** process sequences step by step, learn long-range dependencies
- **Temporal Convolutional Networks:** apply 1D convolutions along time axis (connects to Week 3 CNNs!)
- **Transformers:** attention over time steps: foundation of models like TimesFM, Chronos
- These shine with lots of data and complex patterns, but gradient boosting is a surprisingly tough baseline



Content

- Time Series
- Recommender Systems
- Recap & Lab



The Recommendation Problem

- You have users, you have items, and most users have interacted with only a tiny fraction of items.
- Goal: predict which unseen items a user would like.
- Why? This is one of the most **commercially impactful** ML applications (Netflix, Amazon, Spotify, YouTube, TikTok...)



Explicit vs Implicit Feedback

Explicit

Star ratings, thumbs up/down, reviews

Clear signal but rare

Implicit

Clicks, watch time, purchase history, skips

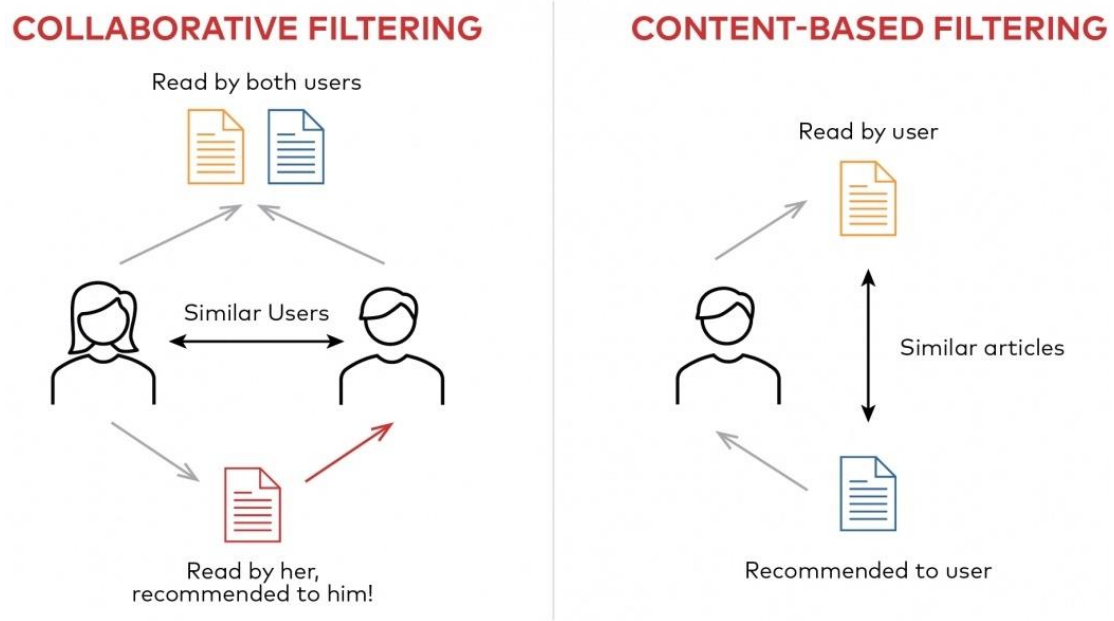
Noisy signal but abundant

- Most real systems rely on implicit feedback



Two Approaches

1. **Content-based filtering:** recommend items similar to what the user already liked, based on item features
2. **Collaborative filtering:** recommend items that similar users liked, based on interaction patterns



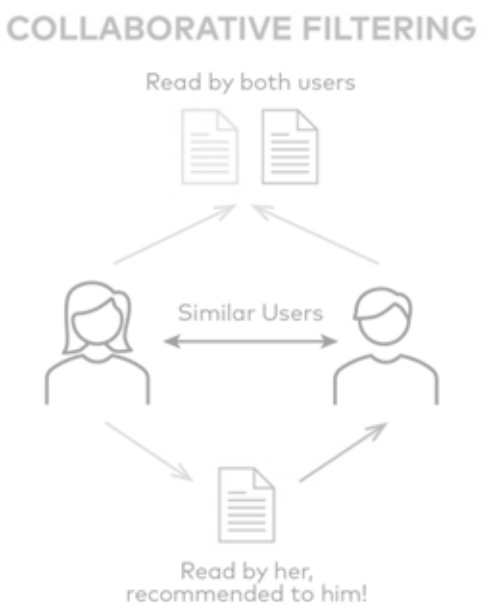


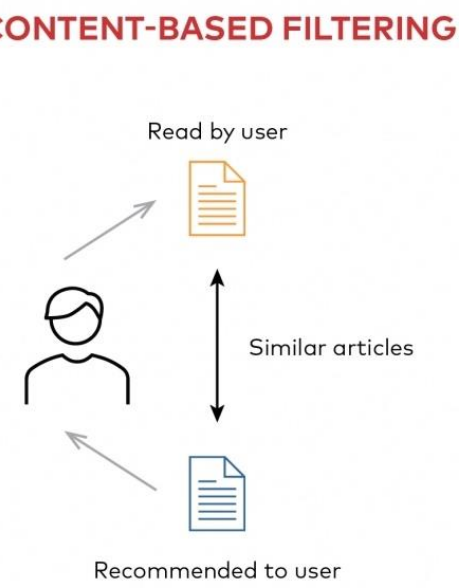
Content-Based Filtering

- Represent each item as a feature vector (genre, tags, description, price, etc.)
- Build a user profile from the items they liked
- Recommend items whose features are closest to the user profile
- Similarity metric: cosine similarity is most common

Strengths: works for new items, transparent recommendations

Weakness: never surprises the user -> stuck in a "filter bubble"

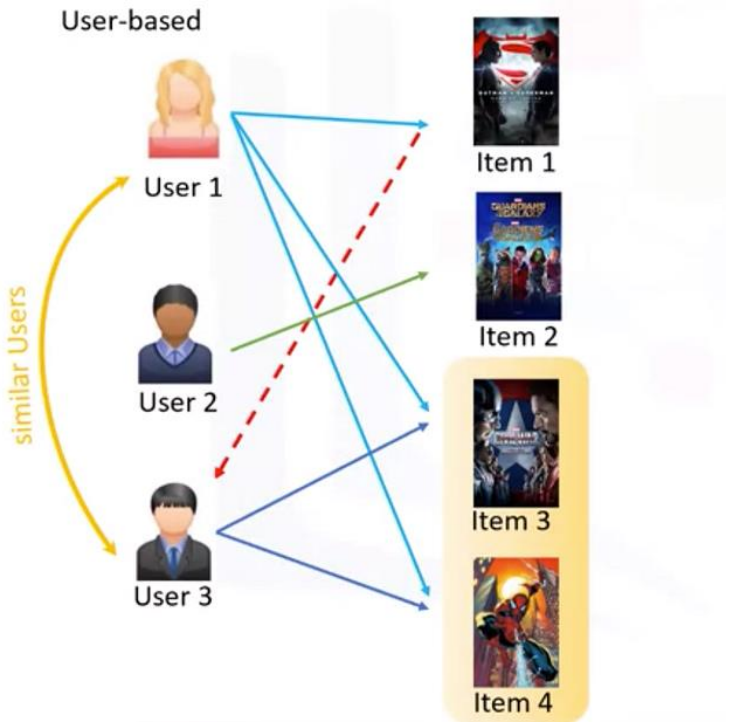




Collaborative Filtering: User-Based

"Users who are similar to you liked these items"

1. Find users with similar rating patterns (nearest neighbors)
2. Look at items those neighbors rated highly
3. Recommend items the target user hasn't seen yet



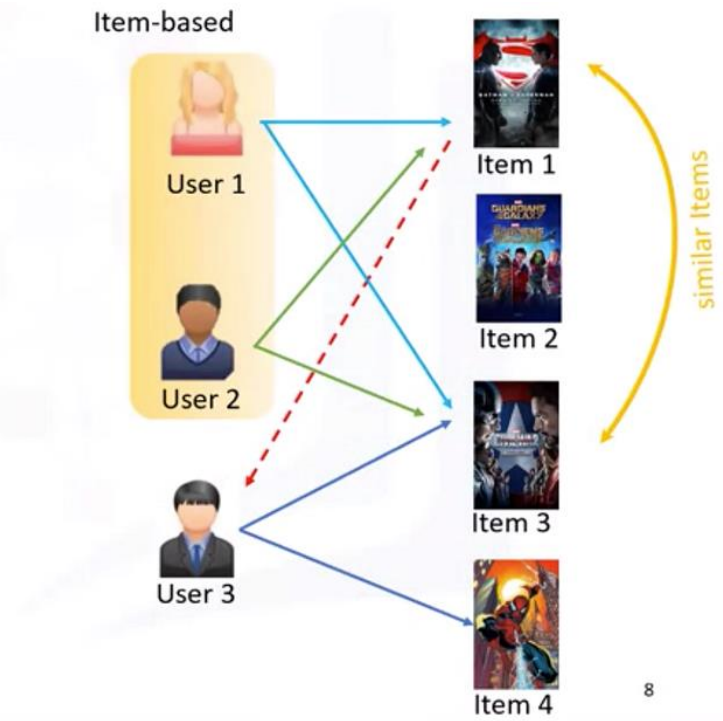


Collaborative Filtering: Item-Based

"Items that are similar to items you liked"

1. Find items with similar rating patterns across users
2. If a user liked item A, recommend items similar to A

In practice often more stable than user-based (item similarities change less than user similarities)





Matrix Factorization

The sparse user-item matrix can be approximated as the product of two smaller matrices: $R \approx U \times I^T$

- R: ratings matrix
- U: user matrix (each user = a vector of latent factors)
- I: item matrix (each item = a vector of latent factors)

	The Matrix	John Wick	Die Hard	Inception	Interstellar	Arrival	The Notebook	Pride & Prejudice	Titanic	Superbad	The Hangover	Bridesmaids
Alice	5.02	3.46	4.64	4.58	4.21	3.44	3.27	3.09	3.63	1.97	3.79	3.68
Bob	3.73	5.42	3.81	4.93	4.59	3.69	2.84	1.55	3.81	3.47	2.43	4.15
Carol	5.20	3.46	4.16	3.82	3.87	4.92	1.31	3.36	1.93	3.87	4.14	4.20
Dave	3.32	1.45	3.31	3.16	3.46	3.56	4.37	5.60	4.07	3.77	5.47	3.81
Eve	1.43	3.28	2.19	3.45	3.83	3.50	4.60	4.30	4.80	5.40	4.18	4.24
Frank	4.40	3.22	4.27	4.39	4.13	3.28	3.78	3.47	4.01	2.34	3.97	3.70
Grace	2.79	3.03	2.93	3.52	3.79	3.88	3.67	4.24	3.90	4.68	4.37	4.15
Hank	3.44	3.88	3.49	4.15	4.12	3.73	3.36	3.18	3.85	3.84	3.64	4.07



Matrix Factorization (Intuition)

- Think of it as finding a hidden low-dimensional space where users and items live together.
- A user's predicted rating for an item = dot product of their two latent vectors.
- If user vector and item vector point in the same direction → high predicted rating
- Foreshadowing: essentially the same idea as embeddings (→ Agentic AI class)



The Cold-Start Problem

What if a user or item is brand new? No interactions yet → collaborative filtering has nothing to work with.

Solutions:

- Fall back to content-based for new items
- Ask new users for initial preferences (onboarding)
- Use popularity-based defaults
- Hybrid systems that combine both approaches



Hybrid Approaches

Most production systems combine multiple strategies:

- **Content-based for cold start, collaborative for established users**
- Use content features as side information in matrix factorization
- Ensemble: blend scores from multiple models
- Deep learning models (e.g., two-tower networks) that combine user features, item features, and interaction patterns in one model



Real-World Considerations

- Scale: millions of users × millions of items → approximate nearest neighbors, candidate generation + ranking pipeline
- Bias: popularity bias (recommending what's already popular), position bias
- Feedback loops: recommending → user clicks → reinforces recommendation
- Ethics: filter bubbles, echo chambers, addictiveness by design



Content

- Time Series
- Recommender Systems
- Recap & Lab



Time Series Recap

- Time = ordered, correlated, non-shuffleable
- Decompose into trend + seasonality + residual
- ML approach: engineer lag, rolling, and calendar features → use gradient boosting
- Always split by time, never randomly
- DL possible but isn't always needed



Recommender Systems Recap

- Content-based: match item features to user preferences
- Collaborative filtering: match user preferences to similar users
- Matrix factorization: learn latent representations of users and items
- Cold-start is the fundamental challenge → hybrid systems
- Production systems blend many approaches + need careful evaluation



ขอบคุณครับ

Thank you!

etienne@cmkl.ac.th

